

SimpleFRP v0.2.0

项目介绍、实现说明与操作指南

版本日期：2026-09-30。目标是快速部署 TCP 反向转发。Linux 提供服务端和客户端，Windows/macOS 提供客户端。安装后自动配置自启，连接依赖服务端输出的一串字符，不再使用旧版密码向导。

本指南不含任何真实服务器凭据或私钥。所有示例中的 IP、端口、隧道编号和连接字符串占位符必须换成你的实际值。先确认目标应用在客户端运行，再将 local 端口指向它。

1. 安装 Linux 服务端

在有公网 IP、使用 systemd 的 Linux 服务器上执行：

```
bash <(curl -fsSL \
  https://github.com/QiaoxiuLi/SimpleFRP/releases/latest/download/install-linux.sh) server
sudo simplefrp run
```

第一条安装并配置开机自启，第二条完成初始化和启动，输出连接字符串。首次自动选择三个不同、非常用且可绑定的端口，分别用于网页、心跳和第一条隧道。重复执行 run 保留现有配置和身份，不重复初始化。公网 IP 自动识别失败时使用 `sudo simplefrp run --public-ip 你的公网IP`。

在云防火墙/安全组中允许输出的心跳端口和 public 隧道端口；无需开放网页端口。软件不会替你修改其它防火墙规则、Nginx 或网站。连接字符串不是加密保险箱，它含有配对凭据，不能公开或提交到 Git。

2. 安装并连接客户端

Linux 客户端：

```
bash <(curl -fsSL \
  https://github.com/QiaoxiuLi/SimpleFRP/releases/latest/download/install-linux.sh) client
sudo simplefrp set <connection-string>
```

macOS 客户端，支持 Apple Silicon 和 Intel：

```
bash <(curl -fsSL \
  https://github.com/QiaoxiuLi/SimpleFRP/releases/latest/download/install-macos.sh)
simplefrp set <connection-string>
```

安装后开一个新终端；当前终端也可以直接使用 `~/local/bin/simplefrp`。

Windows 10/11 x64 客户端，在普通 PowerShell 中执行：

```
& ([scriptblock]::Create((Invoke-RestMethod `
  'https://github.com/QiaoxiuLi/SimpleFRP/releases/latest/download/install-windows.ps1'))
simplefrp set <connection-string>
```

用服务端输出的整串字符替换 `<connection-string>`，不要保留尖括号。双方自动建立第一条隧道。

Windows/macOS 自动弹出状态终端，关闭它不会停止转发。首次自动分配的 local 端口是目标占位端口，软件不会启动你的应用。

3. 对接已有应用

```
simplefrp state
simplefrp 0 local 3000
```

使用 state 显示的实际隧道编号；不保证总是 0。上述命令将客户端本机 3000 端口作为目标。访问服务端的公网 IP 和该隧道的 public 端口，即访问客户端应用。local 与 public 不必一致，local 已被你的应用监听是正确的，不是冲突。如果应用没有监听 127.0.0.1:local 端口，控制连接在线也无法转发应用数据。

Linux 默认系统安装的管理命令需要 sudo。local 可为 1-65535；服务端监听端口为 1025-65535。自动选择会避开常用端口；手动指定可使用空闲的常用非特权端口。

4. 隧道操作

```
simplefrp next
simplefrp 0 public 35001
simplefrp 0 local 3000
simplefrp delete 0
```

next 新建一条隧道，不是新增一个独立端口。public/local 命令修改编号对应的现有隧道，客户端和服务端都可执行。端口冲突、客户端无法保存或离线时，修改失败并保留原映射。客户端只能改自己的隧道；服务端可以删除离线隧道。

服务端网页和心跳端口也可修改，但这不创建隧道：

```
sudo simplefrp port dashboard 29181
sudo simplefrp port heartbeat 29182
```

在线客户端会接收新心跳端口并重连，旧端口保留约 30 秒过渡。离线客户端需要复制新的连接字符串，再执行 set。请为新的心跳/public 端口配置相应的网络放行规则。

5. 状态与网页

```
simplefrp state
simplefrp state --json
simplefrp state --watch
```

显示隧道编号、public/local 端口、实时 B/s 和累计字节数；心跳端口单独显示。流量是双向应用数据之和，不包含 TCP/TLS 开销。断线时显示离线和缓存累计值，速度为零，不伪装成实时数据。服务端重启后保留映射、身份和累计流量；客户端自动重连，不会新建额外隧道。

只有 Linux 服务端有网页。网页仅监听本机，纯只读，不要求设置网页密码，也没有配置按钮。从自己的电脑建立 SSH 转发：

```
ssh -L 18080:127.0.0.1:DASHBOARD_PORT user@SERVER_IP
```

将大写占位内容替换成实际端口、用户名和 IP，在本机浏览器打开 <http://127.0.0.1:18080>。无需也不应为了这个网页修改其它网站的 Nginx 配置。

6. 离线安装、更新与卸载

从 Release 下载符合 CPU/系统的包并校验 checksums.txt。Linux 可以将 RPM/DEB 和安装脚本上传后执行：

```
bash install-linux.sh server --package ./simplefrp-server-linux-amd64.rpm
sudo simplefrp run
```

客户端将 server 和包名中的 server 改为 client。Linux 两种角色不能在同一个默认安装目录同时安装。

Windows 解压 ZIP 后双击 install.cmd 或运行 install.ps1；macOS 解压 TAR.GZ 后执行 sh install.sh。v2 更新可再次运行安装器，会停止已确认归属的旧进程、替换文件并复用配置。

v0.1 与 v0.2 不兼容，旧版 pwd/setup/set server 不再保留。安装器拒绝覆盖旧配置。请先按 BACKUP.md 做私有备份，明确卸载旧版后安装 v2，并用新的字符串重新配对。

```
sudo simplefrp uninstall
```

Linux 使用上面的 sudo 命令；Windows/macOS 客户端使用 simplefrp uninstall。卸载停止所属进程，删除所属自启项、运行文件及自己添加的 PATH 内容。Windows 程序文件会在卸载命令退出后短暂延迟删除。在线客户端还会解除服务端的隧道与身份；服务端不可达时，本地仍卸载并提示后续检查服务端残留映射。其它软件、其它 PATH 项、用户另存的备份不删除。此次验收不执行重启/登录自启测试，但安装器仍配置自启。

7. 排错与恢复

连接失败先检查两端均为 v2、心跳端口已放行、服务端 IP 正确、连接字符串完整。转发失败再检查客户端目标应用是否运行及 local 端口是否正确。不要为了排错停止其它项目。身份校验失败时不能关闭 TLS 验证，应确认连接的确为目标服务端，并重新取得它的正确字符串。运行配置含凭据和私钥，备份必须保持私有；不能上传 GitHub。源码和运行状态是两类备份，分别参见 BACKUP.md。

8. 实现说明

初始化只执行一次。服务端使用加密随机数生成服务端身份、邀请密钥、管理密钥和 Ed25519 证书，自动探测可用的三个端口并保存 TOML 配置。再次运行复用身份和端口，不使已有客户端失效。实际监听绑定是最后的占用校验；碰到其它应用占用不会结束那个应用。

SF2 字符串使用双方相同的 Base62 编解码算法，包含版本、IPv4/IPv6、公网心跳端口、服务端身份、邀请密钥和证书 SHA-256 指纹，还附带校验值用于发现复制损坏。IPv4 字符串约 95 个字符。它只是编码而非加密，拥有它的人可以申请配对，必须像密码一样保密。

客户端首先校验固定的证书指纹，再通过 TLS 1.3 连接。服务端为每个客户端分配独立的 ID 和密钥，不把邀请密钥作为所有客户端永久共享的身份。控制请求使用完整 JSON 的 HMAC-SHA256、时间戳和随机 nonce 防止篡改和重放。身份、隧道和待处理连接数量有上限。

外部 TCP 请求到达 public 端口后，服务端通过控制通道通知所属客户端，客户端再建立独立的 TLS 数据连接，并连接自己的 127.0.0.1:local 端口。双方双向复制数据并保留 TCP 半关闭语义。隧道加密不代替应用端 HTTPS，public 应用协议是否加密由被转发的应用决定。

服务端统一管理所有隧道编号和映射。修改 public 前先绑定新端口，修改配置时要求在线客户端保存并 ACK。失败保留旧映射；成功后释放旧监听。服务端允许删除离线隧道，重新连接时以服务端状态为准。客户端卸载时会请求删除自己的服务端隧道和身份；服务端不可达时会明确提示未确认的远端清理。

bbolt 保存映射、身份、编号计数和累计流量，正常重启后恢复。流量统计使用原子计数器，每秒采样速率。关闭服务时等待所属连接和工作协程退出，再关闭存储。客户端断线重连复用身份和隧道，不重新创建隧道；缓存累计值不会被离线的零值覆盖。

网页由 Go embed 内嵌 HTML/CSS/JavaScript 和 net/http 提供，不依赖 CDN、Node.js 或 Nginx。浏览器定时读取唯一的只读状态接口，写方法被拒绝。网页只绑定回环地址，不要求网页密码，也不能改配置。

Windows/macOS 状态窗口与后台转发进程独立，结束状态查看进程后转发仍应工作。

安装收据记录所属二进制、角色、PATH 修改和服务账户归属。自启文件带有安装标识，进程记录同时包含 PID、启动时间、可执行路径和命令摘要。卸载先验证归属，避免通过同名进程或过期 PID 误删其它软件。Windows 使用延迟自删除，macOS 不删除共享的 .local/bin 目录，Linux 包管理器只负责删除自己的包文件。

9. 依赖与构建

运行依赖是编译进入二进制的 Go 库，不需要用户额外安装 Python、Node.js、Docker 或数据库。Cobra 1.8.1 负责命令解析；go-toml/v2 2.2.2 负责结构化 TOML；bbolt 1.3.11 负责持久化和进程锁；x/sys 0.44.0 负责平台系统调用和 Windows 终端模式。Cobra 还使用 pflag 和 Windows mousetrap。完整许可证随包提供在 THIRD_PARTY_NOTICES.md。

TLS、Ed25519、HMAC、JSON、TCP、HTTP 和随机数使用 Go 标准库。旧版仅用来读取版本号的上游 FRP 依赖及密码/Argon2 辅助代码已经移除，本项目不与 frpc/frps 线协议互通，也没有旧协议兼容开关。源码最低 Go 1.25，发布构建使用 Go 1.27.1。GoReleaser v2/nFPM 生成 DEB/RPM、ZIP、TAR.GZ 和校验文件。ReportLab、FontTools、Poppler 与嵌入的 Noto Sans SC 字体只用于生成和检查本 PDF，不是软件运行依赖。字体来自 Google Fonts，采用 SIL Open Font License。

Linux 需要 systemd 和 root/sudo 安装权限；Windows 使用内置 PowerShell 5.1、WScript.Shell 和当前用户启动目录；macOS 使用 launchd 与 Terminal。Windows/macOS 的自启是用户登录自启，不是无人登录时的系统服务。此次用户要求排除重启/登录自启验收，但安装仍应创建对应配置。

10. 备份、恢复与版本边界

GitHub 保存源码、提交、标签和 Release 安装包。git bundle 可作离线源码备份，并通过 git bundle verify 检查。拉取前先检查本地修改，使用 git pull --ff-only，不强制覆盖工作区。恢复源码不等于恢复服务端身份，后者还需要私有的配置、证书和数据库备份。

对运行数据备份时只停止 SimpleFRP 自己的服务，再归档它的配置和 bbolt 文件；不要将正在写入的数据库当成一致性快照。备份目录权限 700、归档权限 600，禁止把私钥、连接字符串、客户端密钥、PID 文件或日志提交到公开仓库。恢复到停止的同协议版本安装，丢弃旧 PID/锁记录，确认权限后重新安装自启元数据并启动。

v0.1 与 v0.2 不兼容，也不自动迁移旧密码配置。需要先保存旧版本二进制/安装包、服务定义和私有运行状态作为回退备份，再明确卸载旧版，安装新版并重新配对。v2 的普通更新保留配置。安装、卸载和测试不得为了本项目修改其它网站、数据库、服务、防火墙规则或终端应用中的其它会话。

仓库：<https://github.com/QiaoxiuLi/SimpleFRP>

发布下载：<https://github.com/QiaoxiuLi/SimpleFRP/releases>

详细操作、源码恢复与实际验收证据分别见 README.md、BACKUP.md 和 docs/RELEASE_ACCEPTANCE.md。